

POSM Project: FPGA MTS Laser-Lock Core

Onboarding Packet

Version 3 Draft — High-Bandwidth Analog-Demodulated MTS Lock with FPGA-Side Robust Autolock

Photonics and Optical Sciences Club at Minnesota (POSM)

Draft date: June 11, 2026

Project mission. Build the FPGA control core for POSM’s rubidium modulation-transfer spectroscopy (MTS) laser lock. The system must scan the laser, capture the demodulated MTS line shape, support user-selected and FPGA-verified lock acquisition, engage a low-latency feedback loop, monitor lock health, and use split fast/slow outputs to stabilize the laser while avoiding actuator saturation.

Required baseline. The required locking bandwidth target is ≥ 1 MHz using an **analog-demodulated** MTS error signal and a deliberately short FPGA fast path.

How to read this packet. Sections 1–5 explain why the system exists and how the MTS signal becomes an FPGA error signal. Sections 6–12 define the FPGA architecture, modules, state machines, safety logic, and register map. Sections 13–16 define verification, repository structure, acceptance criteria, and the reading list.

Contents

1	Start Here: What Problem Are We Solving?	4
1.1	The one-sentence version	4
1.2	Why a laser needs to be locked	4
1.3	What is an error signal?	4
1.4	What is modulation-transfer spectroscopy doing for us?	5
1.5	What is a lockbox?	5
1.6	What are you building	5
1.7	How the system behaves during use	6
2	System-Level Decision Summary	6
3	MTS Control Loop: From Photodiode to Laser Current	7
3.1	The loop as a block diagram	7
3.2	Purpose of each physical block	8
3.3	Why analog demodulation is used first	8
3.4	CTL200 modulation mapping	9
4	Signal and Waveform Math	9
4.1	Photodiode RF signal	9
4.2	Mixer demodulation	10
4.3	The error signal near the selected lock point	10
4.4	Negative feedback in one equation	10
5	POSM Hardware Interfaces	11
5.1	RF generation and LO phase control	11
5.2	ADC abstraction	11
5.3	DAC abstraction	12
5.4	Analog boundary assumptions	12
6	What the FPGA Must Do	12
6.1	Category A: the fast feedback path	12
6.2	Category B: acquisition, supervision, diagnostics	12
7	Fast-Path Rules	13
7.1	Allowed in the fast path	13
7.2	Not allowed in the fast path	13
7.3	Latency accounting	14
8	FPGA Module Deep Dive	14
8.1	Module list	14
8.2	<code>adc_formatter.sv</code>	15
8.3	<code>adc_guard.sv</code>	15
8.4	<code>error_calc.sv</code>	15
8.5	<code>pi_core.sv</code>	16
8.6	<code>optional_compensator.sv</code>	16
8.7	<code>output_limiter.sv</code> and <code>fault_gate.sv</code>	17
8.8	<code>ramp_scan.sv</code>	17
8.9	<code>trace_capture.sv</code>	17

8.10	<code>robust_autolock.sv</code>	17
8.11	<code>slow_recenter.sv</code>	18
8.12	<code>lock_watch.sv</code>	18
9	Acquisition and Robust Autolock Workflow	18
9.1	State sequence	18
9.2	Detailed workflow	19
9.3	Handoff rules	19
10	Safety and Fault Architecture	19
10.1	Fault sources	19
10.2	Fault response	20
11	Canonical Register Map	20
11.1	Global control/status	20
11.2	ADC configuration	21
11.3	Error calculation	21
11.4	Fast controller	21
11.5	Slow controller / recentering	22
11.6	Ramp / scan generator	22
11.7	Trace capture	23
11.8	Robust autolock descriptor	23
11.9	Lock check / lock watch	23
11.10	DAC configuration and safety	24
11.11	Latency instrumentation	24
12	Simulation and Verification Plan	24
12.1	Required fake models	24
12.2	Required system simulation	25
12.3	Required plots	25
12.4	Module-level tests	25
13	Repository Structure	26
14	Acceptance Checklist	27
15	Papers, Datasheets, and Learning Resources	27
15.1	Spectroscopy locking and FPGA lockbox systems	27
15.2	FPGA arithmetic, PI, filters, and verification	28
15.3	Device datasheets and hardware interface references	29
15.4	POSM-specific documents	30
16	Final Notes	30

1 Start Here: What Problem Are We Solving?

This packet assumes no background in atomic physics, laser spectroscopy, or laboratory lockboxes. The FPGA work is digital design, but the code only makes sense if the developer understands what the signals represent. This section gives the minimum physical and control-systems background needed before writing RTL.

1.1 The one-sentence version

We are building an FPGA-based control system that keeps a laser's frequency fixed to a chosen rubidium spectroscopy feature by continuously measuring a voltage error signal and applying a correction voltage back to the laser controller.

Plain-English control loop

If the laser frequency drifts too high, the spectroscopy signal tells us “too high.” The FPGA responds by pushing the laser back down. If the laser frequency drifts too low, the FPGA pushes it back up. When the laser is exactly at the chosen lock point, the error signal is approximately zero.

1.2 Why a laser needs to be locked

In a rubidium MOT and spectroscopy setup, the laser frequency must sit at a very specific detuning relative to an atomic transition. The word *detuning* means

$$\Delta(t) = \nu_{\text{laser}}(t) - \nu_{\text{atom}}, \quad (1)$$

where ν_{laser} is the laser frequency and ν_{atom} is the relevant atomic resonance frequency. The experiment cares about $\Delta(t)$, not just the absolute laser current or temperature.

Real lasers drift because of temperature, current noise, acoustic noise, mechanical motion, and electronics noise. If the frequency drifts, the MOT becomes worse or stops working. A laser lock is a feedback system that constantly corrects the laser so that

$$\Delta(t) \rightarrow \Delta_0, \quad (2)$$

where Δ_0 is the selected operating point.

1.3 What is an error signal?

A feedback controller needs a number that tells it which direction to push. That number is the **error signal**. For a good spectroscopy lock, the error signal has a zero crossing at the desired laser frequency and has opposite signs on either side:

$$e(\Delta) \approx S(\Delta - \Delta_0) \quad \text{near the chosen lock point.} \quad (3)$$

Here S is the slope of the error signal. If $S > 0$, then positive error means the laser is on one side of the feature; negative error means it is on the other. If the slope sign is reversed, the controller polarity must also be reversed. This is why the FPGA needs a runtime-configurable `ERROR_POLARITY` bit.

1.4 What is modulation-transfer spectroscopy doing for us?

Modulation-transfer spectroscopy (MTS) is an optical/RF technique that produces a clean dispersive error signal from a rubidium vapor cell. For this FPGA project, the detailed nonlinear optics are less important than the signal-chain result:

- the laser passes through a rubidium spectroscopy setup;
- the optical setup produces a photodiode signal containing a component at the EOM modulation frequency, around 10 MHz;
- that RF component changes phase and amplitude as the laser scans across an atomic feature;
- mixing that RF signal with a coherent 10 MHz local oscillator produces a baseband dispersive error signal;
- the FPGA samples that baseband error signal and drives it to zero.

The important thing: the FPGA is *not* trying to understand atoms directly. It sees voltages. Its job is to scan, recognize the selected voltage feature, engage feedback safely, and keep the corrected error near zero.

1.5 What is a lockbox?

A lockbox is the electronics/control system that sits between a measured error signal and an actuator. In our case:

```
measured MTS error voltage ---> FPGA lockbox ----> correction voltage to laser controller
```

Traditional analog lockboxes use op-amps, knobs, analog PI filters, and analog switches. Linien/PyRPL-style digital lockboxes use ADCs, FPGA logic, DACs, and software. POSM is building a custom hybrid: analog demodulation creates the MTS error quickly, then the FPGA performs scanning, autolock, PI control, safety, and diagnostics.

1.6 What are you building

You are *not* being asked to align the optics, design the mixer PCB, design the DAC output stage, or tune rubidium spectroscopy on day one. Your core deliverable is the FPGA logic that operates on already-digitized signals.

The FPGA receives:

- ADC_CH0: demodulated MTS error signal, used for the real lock;
- ADC_CH1: raw RF monitor signal, used for diagnostics and future digital demodulation;
- configuration registers from the PC;
- optional interlock/fault inputs.

The FPGA outputs:

- DAC_FAST: fast correction to the CTL200 high-frequency/current modulation path;
- DAC_SLOW: slow scan, centering, and drift correction to the CTL200 DC modulation path;
- status, fault, trace, and lock-health data back to the PC.

1.7 How the system behaves during use

The lock has two very different jobs: **acquisition** and **feedback**. Acquisition finds the right spectroscopy feature. Feedback keeps the laser there.

1. **Wide scan:** the FPGA ramps the slow DAC so the laser scans across multiple spectroscopy features.
2. **Trace capture:** the FPGA records pairs of (`slow_dac_code`, `mts_error`) so the PC can plot the line shape.
3. **User selection:** the user selects the desired MTS feature on the plot, similar to Linien's click/drag workflow.
4. **Feature descriptor:** the PC computes a compact description of the selected feature: window, extrema, amplitude, width, slope sign, zero-crossing position, and DC offset.
5. **Robust FPGA autolock:** the FPGA zoom-scans, verifies that the selected peak shape is present, moves to the lock point, and engages the controller at deterministic timing.
6. **Fast lock:** the FPGA uses a short PI feedback path to drive the error toward zero through `DAC_FAST`.
7. **Slow recentering:** after lock, the slow DAC gently moves the operating point so the fast DAC remains near its center instead of railing.

Important

The GUI and PC are allowed to help choose the line and configure parameters. They are not allowed to sit inside the 1 MHz feedback path.

2 System-Level Decision Summary

The following decisions are considered frozen unless hardware testing forces a change. They are written here in direct language so the RTL, PC scripts, and analog hardware can be developed against the same system model.

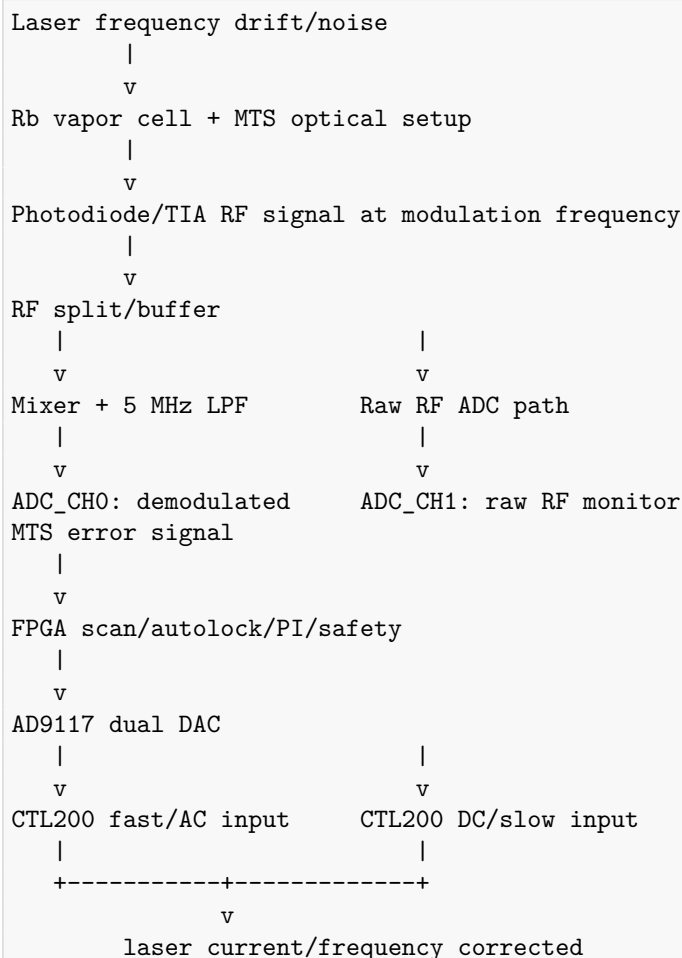
Topic	Decision
Primary error signal	Analog-demodulated MTS error on <code>ADC_CH0</code> . This is the signal used by the required high-bandwidth lock.
Raw RF monitor	<code>ADC_CH1</code> samples the raw photodiode RF branch for diagnostics and future digital demodulation experiments. It is not required for the first high-bandwidth lock.
RF source	AD9959 provides one coherent channel for the EOM drive and one coherent channel for the mixer LO. The PC controls the AD9959 and FPGA as the control-plane middleman.
Mixer LPF	A 5 MHz low-pass after the mixer. This is a mixer-cleanup filter, not a slow display filter.
ADC candidate	A dual simultaneous ADC such as ADC3664 so <code>CH0</code> and <code>CH1</code> can be captured together.
DAC candidate	AD9117 dual DAC. Use one output for fast correction and the other for slow scan/recentering.

Laser controller	Koheron CTL200. DC modulation input supports DC–10 MHz; AC modulation input is for modulation above 100 kHz.
Fast feedback path	ADC_CH0 -> formatter -> guard -> error_calc -> PI -> limiter/fault_gate -> DAC_FAST.
Controller	PI core plus optional low-latency compensation. No naked derivative term in the baseline controller.
Acquisition	Robust FPGA-side autolock inspired by Linien: user selects a feature, PC computes a descriptor, FPGA verifies peak shape and engages lock deterministically.
Slow output	Before lock: scan and center. After lock: slow recentering/drift correction to keep the fast DAC near midscale.
Safety	Hard programmable limits, startup safe codes, sticky faults, explicit fault clear, ADC/DAC/CTL200 protection.

3 MTS Control Loop: From Photodiode to Laser Current

This section follows the signal from the photodiode all the way back to the laser controller. The goal is to explain what each block contributes and what waveform the FPGA eventually receives.

3.1 The loop as a block diagram



3.2 Purpose of each physical block

Block	Example POSM part / plan	Purpose
Rb MTS optics	POSM MTS setup	Turns laser detuning into a spectroscopy signal. When scanned, the final demodulated signal has a characteristic dispersive line shape with a zero crossing.
Photodiode/TIA	FDS010	Converts optical power variations into an electrical RF voltage. This RF voltage contains information about the laser detuning.
AD9959	AD9959 DDS	Produces coherent RF channels: one for EOM modulation and one for mixer LO phase control. The two channels share a clock, so their relative phase can be controlled digitally.
EOM drive	Custom Circuit	Applies phase/frequency modulation at approximately 10 MHz. This modulation is what allows MTS to produce a clean dispersive error signal.
Mixer	TBD	Multiplies the photodiode RF signal with a coherent LO. This shifts the useful information from 10 MHz down to baseband.
Mixer LPF	5 MHz LPF baseline	Removes the high-frequency mixer product near $2f_m$ while preserving MHz-class baseband error dynamics.
ADC	ADC3664	Digitizes ADC_CH0 demodulated error and ADC_CH1 raw RF monitor.
FPGA	Ultra96V2 (7020)	Performs scan generation, trace capture, robust autolock, PI feedback, slow recentering, safety, and register-controlled operation.
DAC	AD9117	Provides two output commands: fast correction and slow scan/recenter.
Laser controller	Koheron CTL200	Converts modulation input voltage into laser-current modulation, which changes the laser frequency.
PC control	Python/GUI scripts	Controls AD9959 phase/frequency/amplitude and FPGA registers, displays trace, and allows line selection.

3.3 Why analog demodulation is used first

There are two ways to recover the MTS error signal:

1. sample the raw RF signal and demodulate digitally inside the FPGA;
2. demodulate with an analog mixer and then sample the baseband error signal.

POSM's required feedback bandwidth is at least 1 MHz. Because delay directly reduces phase margin, the first required architecture uses analog demodulation so the FPGA fast path can remain short. The

raw RF channel is still sampled on ADC_CH1 for diagnostics and future digital-demod experiments, but it is not required for the first MHz-class lock.

3.4 CTL200 modulation mapping

The CTL200 has two modulation paths. The exact hardware version must be confirmed in the lab, but the v5+ documentation describes:

- a DC modulation input from DC to 10 MHz, selectable gain, $50\ \Omega$ input impedance;
- an AC modulation input for modulation above 100 kHz, with approximately 20 mA/V modulation gain.

This means the physical mapping should be understood as split-actuator control:

$$\text{DAC_FAST} \rightarrow \text{CTL200 AC/high-frequency modulation input}, \quad (4)$$

$$\text{DAC_SLOW} \rightarrow \text{CTL200 DC modulation input for scan, centering, and slow correction}. \quad (5)$$

Important

The AC path cannot carry true DC integral correction. The low-frequency/integral authority must live on the DC/slow path. The fast path handles high-frequency correction, while the slow path handles scan, centering, and long-term drift.

4 Signal and Waveform Math

The FPGA developer does not need a full MTS derivation, but should understand how a 10 MHz photodiode signal becomes a baseband error signal.

4.1 Photodiode RF signal

A simplified model for the photodiode voltage is

$$v_{\text{PD}}(t) = V_{\text{DC}}(\Delta) + A(\Delta) \cos(\omega_m t + \phi_{\text{MTS}}(\Delta)) + n(t), \quad (6)$$

where:

- $\omega_m = 2\pi f_m$ is the EOM modulation angular frequency, approximately 10 MHz;
- $A(\Delta)$ is the detuning-dependent RF amplitude;
- $\phi_{\text{MTS}}(\Delta)$ is the detuning-dependent RF phase;
- $n(t)$ is noise.

The important part is the RF term at f_m . As the laser scans across the rubidium feature, the amplitude and phase of this RF term change. The mixer converts that phase/amplitude information into a baseband voltage.

4.2 Mixer demodulation

The mixer LO is generated coherently from another AD9959 channel:

$$v_{\text{LO}}(t) = \cos(\omega_m t + \phi_{\text{LO}}). \quad (7)$$

The mixer approximately multiplies the photodiode RF component by the LO:

$$v_{\text{mix}}(t) \propto A(\Delta) \cos(\omega_m t + \phi_{\text{MTS}}) \cos(\omega_m t + \phi_{\text{LO}}). \quad (8)$$

Using

$$\cos(a) \cos(b) = \frac{1}{2} [\cos(a - b) + \cos(a + b)], \quad (9)$$

we get two terms:

$$v_{\text{mix}}(t) \propto \frac{A(\Delta)}{2} \cos(\phi_{\text{MTS}}(\Delta) - \phi_{\text{LO}}) + \frac{A(\Delta)}{2} \cos(2\omega_m t + \phi_{\text{MTS}}(\Delta) + \phi_{\text{LO}}). \quad (10)$$

The first term is baseband. The second term is near $2f_m$, roughly 20 MHz for a 10 MHz modulation. The 5 MHz LPF removes the high-frequency term and leaves the useful error signal:

$$e_{\text{raw}}(\Delta) \approx K A(\Delta) \cos(\phi_{\text{MTS}}(\Delta) - \phi_{\text{LO}}) + e_{\text{offset}}. \quad (11)$$

The LO phase ϕ_{LO} is important. Changing it rotates which quadrature of the RF signal is measured. POSM will tune this phase through the AD9959 channel phase offset using a PC script.

4.3 The error signal near the selected lock point

After the user selects a feature and the software/FPGA determines the correct zero crossing, the useful local model is simply

$$e_{\text{raw}}(\Delta) \approx e_{\text{offset}} + S(\Delta - \Delta_0), \quad (12)$$

where:

- e_{offset} is a DC/background offset;
- S is the local slope;
- Δ_0 is the selected lock point.

The FPGA subtracts the offset and applies the selected polarity:

$$e[n] = p(x_{\text{ADC0}}[n] - \text{ERROR_OFFSET} - \text{ERROR_SETPOINT}), \quad (13)$$

where $p = +1$ or $p = -1$ depending on the selected slope sign. The controller tries to make

$$e[n] \rightarrow 0. \quad (14)$$

4.4 Negative feedback in one equation

A simple discrete-time PI controller can be written as

$$I[n + 1] = I[n] + K_i e[n], \quad (15)$$

$$u[n] = K_p e[n] + I[n]. \quad (16)$$

Here $u[n]$ is the correction command. In the real FPGA design, this must be fixed-point, saturating, limit-protected, and carefully handed off from scan mode. The idea is still simple: if the error is nonzero, the controller changes the laser current in the direction that reduces the error.

Important

The sign matters. If the controller polarity is wrong, the loop becomes positive feedback and runs away. This is why the selected slope sign and the `ERROR_POLARITY` register are not optional.

5 POSM Hardware Interfaces

5.1 RF generation and LO phase control

Use the AD9959 as the coherent RF source:

```
AD9959 CH A: 10 MHz EOM drive, phase = 0 deg
AD9959 CH B: 10 MHz mixer LO, phase = phi_demod
AD9959 CH C: reserved for 6--120 MHz serrodyne/other RF task
AD9959 CH D: spare or future use
```

The PC controls both AD9959 and FPGA:

```
PC script / GUI
|-- set AD9959 EOM frequency/amplitude
|-- set AD9959 LO frequency/amplitude/phase
|-- command FPGA scan/autolock/lock registers
|-- read FPGA traces/status
```

This is acceptable because LO phase optimization is not in the MHz feedback loop. A typical phase-optimization routine is:

1. set LO phase ϕ ;
2. run a scan;
3. capture the MTS trace from FPGA;
4. score slope, peak-to-peak amplitude, symmetry, offset, and noise;
5. choose the best ϕ .

5.2 ADC abstraction

The gateway must not be written around a single ADC chip too early. Internally, expose a clean stream:

```
input logic          clk;
input logic          rst;
input logic signed [ADC_W-1:0] adc_ch0_sample; // demod MTS error
input logic signed [ADC_W-1:0] adc_ch1_sample; // raw RF monitor
input logic          adc_valid;
input logic          adc_ch0_overrange;
input logic          adc_ch1_overrange;
```

Board-specific ADC deserialization, SLVDS/CMOS capture, input formatting, bit slips, and timing constraints belong in `board_top.sv`, not in the lock core.

5.3 DAC abstraction

Internally, the FPGA emits two signed actuator commands:

```
output logic signed [DAC_W-1:0] dac_fast_code;
output logic signed [DAC_W-1:0] dac_slow_code;
output logic                    dac_valid;
```

Board-specific AD9117 formatting, clocking, output interface timing, and output-enable behavior belong in a DAC wrapper.

5.4 Analog boundary assumptions

The FPGA packet assumes:

- ADC_CH0 is a signed digitized version of the analog-demodulated MTS error signal.
- ADC_CH1 is a signed digitized raw RF monitor signal.
- DAC_FAST maps to a bounded voltage at the CTL200 fast/AC modulation input.
- DAC_SLOW maps to a bounded voltage at the CTL200 DC modulation input.
- hardware scaling and clamps must be chosen so full-scale DAC codes cannot command unsafe current swings.

6 What the FPGA Must Do

The FPGA has two different categories of work.

6.1 Category A: the fast feedback path

This path is latency-critical:

```
ADC_CH0
-> adc_formatter
-> adc_guard
-> error_calc
-> pi_core
-> optional_compensator, bypassed by default
-> output_limiter
-> fault_gate
-> dac_fast_formatter
-> DAC_FAST
```

The fast path must have deterministic latency and must not depend on the PC, GUI, network, trace capture, feature fitting, or autolock search logic.

6.2 Category B: acquisition, supervision, diagnostics

These blocks are important but not allowed to slow the fast path:

- ramp/scan generation;
- trace capture;
- robust autolock;

- lock check and lock watch;
- slow recentering;
- raw RF capture/diagnostics;
- register map and PC control;
- fault logging and sticky status bits.

Important

The fast path and the acquisition/supervision path must be architecturally separate. A feature can be useful and still be forbidden from the MHz feedback path.

7 Fast-Path Rules

7.1 Allowed in the fast path

- ADC formatting and sign conversion.
- ADC guard/validity checks that do not add unnecessary latency.
- Offset/setpoint subtraction and polarity correction.
- PI control.
- Optional low-latency compensator, bypassed by default.
- Output limiter and fault gate.
- DAC formatting.

7.2 Not allowed in the fast path

- GUI or PC interaction.
- USB/Ethernet/network transactions.
- Trace capture or BRAM readout.
- Feature fitting or peak detection.
- Digital demodulation for the required v1 lock.
- FFTs, PSDs, or spectrum analyzers.
- Long moving averages or slow smoothing filters.
- ML or POSMq inference.

7.3 Latency accounting

At 1 MHz, pure delay creates phase lag:

$$\phi_{\text{delay}} = -360^\circ f \tau. \quad (17)$$

Examples:

Delay τ	Phase lag at 1 MHz	Meaning
50 ns	18°	excellent
100 ns	36°	still reasonable
125 ns	45°	serious but manageable
200 ns	72°	dangerous without plant margin
320 ns	115°	likely too much for aggressive MHz locking

Every module in the fast path must declare its `valid-in -> valid-out` latency in cycles.

8 FPGA Module Deep Dive

8.1 Module list

Module	Purpose
<code>adc_formatter.sv</code>	Converts ADC output coding to signed internal samples.
<code>adc_guard.sv</code>	Detects overrange, missing valid, stuck-at-rail, and invalid input conditions.
<code>error_calc.sv</code>	Applies offset subtraction, setpoint subtraction, and polarity inversion.
<code>pi_core.sv</code>	Implements fixed-point PI control with saturation and anti-windup.
<code>optional_compensator.sv</code>	Bypassable low-latency lead/lag or first-order compensator for later loop shaping.
<code>output_limiter.sv</code>	Applies programmable hard limits and saturation flags.
<code>fault_gate.sv</code>	Overrides actuator outputs to safe codes on fault or disable.
<code>dac_fast_formatter.sv</code>	Converts signed internal fast command to AD9117/board-specific code.
<code>dac_slow_formatter.sv</code>	Converts signed internal slow command to AD9117/board-specific code.
<code>ramp_scan.sv</code>	Generates wide and zoom scan ramps for <code>DAC_SLOW</code> .
<code>trace_capture.sv</code>	Captures scan-code/error-signal pairs for display and line selection.
<code>robust_autolock.sv</code>	Performs FPGA-side selected-feature recognition and deterministic lock engagement.
<code>lock_fsm.sv</code>	Owens system states: idle, scan, zoom, arm, lock, watch, relock, hold, fault.
<code>slow_recenter.sv</code>	Slowly adjusts <code>DAC_SLOW</code> to keep <code>DAC_FAST</code> near center after lock.
<code>lock_watch.sv</code>	Checks error, saturation, output rails, and health after lock.
<code>register_bank.sv</code>	Provides the PC/GUI/POSMq software contract.
<code>latency_probe.sv</code>	Measures or reports module and fast-path latency.

8.2 adc_formatter.sv

Purpose. Convert physical ADC data format into signed two's-complement internal samples. If the ADC output is offset binary, convert immediately. If the ADC is already two's complement, sign-extend as needed.

```
// Offset-binary to signed two's complement, conceptual
assign signed_sample = adc_raw ^ (1 << (ADC_W-1));
```

Tests. Full-scale negative, midscale zero, full-scale positive, sign extension, invalid samples.

8.3 adc_guard.sv

Purpose. Prevent the controller from acting on obviously invalid ADC data. It should detect:

- ADC overrange;
- stuck high/low;
- missing `adc_valid`;
- too many consecutive samples outside configured bounds.

Output. Status bits and fault candidates. The guard may raise a fault if enabled in `FAULT_ENABLE`.

8.4 error_calc.sv

Purpose. Convert the corrected ADC sample into a signed control error.

$$e[n] = p(x[n] - x_{\text{offset}} - x_{\text{setpoint}}), \quad p \in \{+1, -1\}. \quad (18)$$

```
module error_calc #(
    parameter int ADC_W = 16,
    parameter int ERR_W = 20
)()
    input  logic clk,
    input  logic rst,
    input  logic signed [ADC_W-1:0] sample_in,
    input  logic sample_valid,
    input  logic signed [ADC_W-1:0] offset,
    input  logic signed [ADC_W-1:0] setpoint,
    input  logic invert_error,
    output logic signed [ERR_W-1:0] error_out,
    output logic error_valid
);
// x_corr = sample_in - offset - setpoint
// error_out = invert_error ? -x_corr : x_corr
endmodule
```

Tests. Offset subtraction, setpoint subtraction, polarity inversion, reset, valid latency, overflow/saturation behavior.

8.5 pi_core.sv

Purpose. Implement fixed-point PI feedback. The core must be deterministic, saturating, and testbench-verified.

$$I[n] = I[n - 1] + K_i e[n], \quad (19)$$

$$u[n] = K_p e[n] + I[n], \quad (20)$$

$$u_{\text{lim}}[n] = \text{clip}(u[n], u_{\text{min}}, u_{\text{max}}). \quad (21)$$

Required features.

- signed fixed-point K_p and K_i ;
- wide internal accumulator;
- programmable output limits;
- anti-windup;
- hold;
- `integrator_reset`;
- `integrator_load` or `output_load` for smooth handoff;
- saturation flags.

Important

Because the CTL200 AC input is not a true DC actuator, the high-frequency path should not be expected to carry all long-term integral correction. The slow/DC path must handle scan, bias, and long-term recentering.

8.6 optional_compensator.sv

Purpose. Provide a place for future low-latency loop shaping without rewriting the fast path. Baseline state is bypass.

Possible future form:

$$y[n] = b_0 x[n] + b_1 x[n - 1] - a_1 y[n - 1]. \quad (22)$$

Rules.

- Bypass by default.
- No long FIRs in the fast path.
- No slow smoothing filters in the fast path.
- Coefficients must be fixed-point documented.

8.7 output_limiter.sv and fault_gate.sv

The limiter clips actuator-facing commands:

$$y = \min(\max(x, y_{\min}), y_{\max}). \quad (23)$$

The fault gate can force output to a safe code regardless of normal controller output.

```
normal operation: output = limited_controller_output
fault/disabled:   output = SAFE_CODE
hold:             output = held_last_safe_code
```

8.8 ramp_scan.sv

Purpose. Generate wide and zoom scans on DAC_SLOW. For v1, triangle scans are sufficient.

```
if direction_up:
    ramp = min(ramp + step, ramp_max)
else:
    ramp = max(ramp - step, ramp_min)
reverse direction at endpoints
```

Tests. No overshoot, correct reversal, programmable bounds, disabled behavior, reset behavior, zoom mode around center/width.

8.9 trace_capture.sv

Purpose. Capture line-shape traces for GUI display and feature selection. A trace sample is a pair:

$$(X_i, Y_i) = (\text{slow scan code}, \text{MTS error sample}). \quad (24)$$

This block is not in the fast feedback path.

8.10 robust_autolock.sv

Purpose. Perform FPGA-side feature confirmation and lock engagement after the user/PC has selected a target feature and sent a compact descriptor. This follows the useful idea from Linien: time-critical lock-point selection should happen on FPGA, not after a PC round trip.

Inputs from PC descriptor.

- selected scan window;
- approximate minimum and maximum positions;
- expected zero-crossing position;
- expected slope sign;
- amplitude threshold;
- width tolerance;
- computed y-offset;
- retry policy.

FPGA behavior.

1. Run zoom scan over selected window.
2. Track local extrema.
3. Verify amplitude, width, order, and slope sign.
4. Move/hold slow DAC near lock position.
5. Load offset, polarity, and controller state.
6. Enable fast feedback on a deterministic sample boundary.

8.11 `slow_recenter.sv`

Purpose. After lock, slowly adjust DAC_SLOW so that DAC_FAST stays near its center code:

$$s[n + 1] = s[n] + K_s (u_{\text{fast}}[n] - u_{\text{fast,center}}) . \quad (25)$$

The update rate should be slow compared with the fast loop.

8.12 `lock_watch.sv`

Purpose. Decide whether the system remains locked. It should monitor:

- absolute error;
- error RMS or peak-to-peak over a diagnostic window;
- fast DAC near rails;
- saturation timeout;
- ADC invalid states;
- sudden control jumps.

9 Acquisition and Robust Autolock Workflow

The desired user experience is Linien-like: see a spectroscopy trace, select a line, zoom in, and lock. The implementation should be POSM-specific and low-latency.

9.1 State sequence

```

IDLE
-> WIDE_SCAN
-> TRACE_READY
-> USER_SELECT
-> ZOOM_SCAN
-> FEATURE_VERIFY
-> ARM_LOCK
-> LOCKED
-> LOCK_WATCH
-> RELOCK_SCAN or FAULT if needed
    
```

9.2 Detailed workflow

1. **Wide scan.** FPGA ramps DAC_SLOW and captures `scan_code`, `error` trace pairs.
2. **User selection.** PC/GUI displays the demodulated MTS trace. The user selects one target feature containing both extrema.
3. **Feature descriptor.** PC computes expected extrema, zero-crossing code, amplitude threshold, width tolerance, y-offset, and slope sign.
4. **Zoom scan.** FPGA scans a narrower window around the selected feature.
5. **Robust verify.** FPGA verifies that the observed feature matches the descriptor.
6. **Arm lock.** FPGA holds/centers the slow output, initializes the PI state, loads offset/polarity, and prepares output limits.
7. **Engage.** FPGA enables feedback on a deterministic sample boundary.
8. **Check lock.** FPGA waits a configured delay and checks error/output metrics.
9. **Watch/recenter.** FPGA remains locked while slow recentering keeps the fast output away from rails.

9.3 Handoff rules

The scan-to-lock transition must not kick the laser. Required controls:

- `fast_output_hold`;
- `slow_output_hold`;
- `integrator_reset`;
- `integrator_load` or `output_load`;
- `synchronized lock_enable`;
- safe initial fast output code.

10 Safety and Fault Architecture

10.1 Fault sources

Fault	Meaning
ADC overrange	ADC reports or appears outside valid range.
ADC stuck high/low	Sample remains near rail for too many samples.
Missing valid	ADC stream stops or <code>adc_valid</code> timeout occurs.
Fast saturation timeout	Fast output is clipped high/low for too long.
Slow saturation timeout	Slow output is clipped high/low for too long.
Fast near rail	Fast output is near configured rail even if not clipped.
Invalid mode	Register mode code is illegal.
Autolock failed	Feature verification or lock engagement fails after retry limit.

Lock check failed	Error/output metrics fail after engagement.
External interlock	User kill, external hardware interlock, clock/PLL fault, etc.

10.2 Fault response

Fault behavior must be explicit:

- disable fast feedback;
- force DAC_FAST to FAST_OUT_SAFE;
- either hold or safe-code DAC_SLOW, depending on configured fault policy;
- set sticky fault bits;
- require explicit clear before re-entering normal operation.

Important

Do not silently auto-recover from safety-critical faults. A relock policy can be automatic, but rail/overrange/interlock faults should require deliberate handling.

11 Canonical Register Map

All addresses below are byte offsets. All registers are 32-bit. Signed values are two's-complement. Reserved bits must read as zero when practical and ignore writes. Every writable configuration value should be readable.

11.1 Global control/status

Addr.	Name	R/W	Description
0x000	VERSION	R	Build/version ID.
0x004	CONTROL	R/W	Global enable, reset, output enable, lock request, trace/autolock enable.
0x008	STATUS	R	State, locked, scanning, saturation, trace ready, fault active.
0x00C	MODE	R/W	Idle, scan, zoom, arm, lock, hold, fault, debug.
0x010	FAULT_STATUS	R	Sticky fault flags.
0x014	FAULT_ENABLE	R/W	Mask enabling faults to force fault state.
0x018	FAULT_CLEAR	W	Write-one-to-clear selected sticky faults.
0x01C	DEBUG_SELECT	R/W	Select internal debug signal.

Suggested CONTROL bits

Bit	Name	Meaning
0	global_enable	Enables non-fault operation.
1	soft_reset	Synchronous soft reset request.

2	outputs_enable	Allows DAC outputs to leave safe code.
3	lock_enable_request	Requests lock engagement.
4	hold_request	Requests hold mode.
5	fault_clear_request	Requests fault clear.
6	integrator_reset	Resets PI integrator.
7	integrator_load	Loads PI/output state for smooth handoff.
8	trace_capture_enable	Enables trace capture.
9	autolock_enable	Enables robust autolock.
10	slow_recenter_enable	Enables slow recentering while locked.
11	adc_test_pattern_enable	Uses internal ADC test pattern.
12	dac_test_pattern_enable	Uses DAC test output codes.

11.2 ADC configuration

Addr.	Name	R/W	Description
0x040	ADC_CONFIG	R/W	ADC format, channel enable, test modes.
0x044	ADC_CHO_OFFSET	R/W	Raw ADC offset for demodulated MTS signal.
0x048	ADC_CH1_OFFSET	R/W	Raw ADC offset for raw RF monitor.
0x04C	ADC_CHO_GAIN	R/W	Optional digital gain for CH0.
0x050	ADC_CH1_GAIN	R/W	Optional diagnostic gain for CH1.
0x054	ADC_CHO_MIN_VALID	R/W	Lower valid threshold for CH0.
0x058	ADC_CHO_MAX_VALID	R/W	Upper valid threshold for CH0.
0x05C	ADC_CH1_MIN_VALID	R/W	Lower valid threshold for CH1.
0x060	ADC_CH1_MAX_VALID	R/W	Upper valid threshold for CH1.
0x064	ADC_GUARD_COUNT	R/W	Bad-sample count before fault candidate.
0x068	ADC_STATUS	R	Overrange, missing valid, stuck-at-rail flags.

11.3 Error calculation

Addr.	Name	R/W	Description
0x080	ERROR_SETPOINT	R/W	Desired lock error, usually zero after offset correction.
0x084	ERROR_OFFSET	R/W	Signal offset from feature analysis, typically $(y_{\max} + y_{\min})/2$.
0x088	ERROR_POLARITY	R/W	Error inversion / slope sign.
0x08C	ERROR_THRESH_LOCKED	R/W	Threshold for locked candidate.
0x090	ERROR_THRESH_UNLOCK	R/W	Larger threshold for declaring unlock.
0x094	ERROR_RMS_WINDOW	R/W	Diagnostic RMS window length. Not fast path.
0x098	ERROR_CURRENT	R	Latest corrected error sample.
0x09C	ERROR_ABS_MAX	R	Peak absolute error since last clear.

11.4 Fast controller

Addr.	Name	R/W	Description
0x0C0	FAST_CTRL_CONFIG	R/W	Enable P/I paths, optional compensator, hold/load behavior.
0x0C4	FAST_KP	R/W	Signed fixed-point proportional gain.
0x0C8	FAST_KI_LOCAL	R/W	Optional local/leaky integral. Often zero or limited for AC path.
0x0CC	FAST_INT_LEAK	R/W	Leaky-integrator coefficient.
0x0D0	FAST_COMP_A1	R/W	Optional compensator coefficient.
0x0D4	FAST_COMP_B0	R/W	Optional compensator coefficient.
0x0D8	FAST_COMP_B1	R/W	Optional compensator coefficient.
0x0DC	FAST_OUT_MIN	R/W	Hard minimum fast DAC code.
0x0E0	FAST_OUT_MAX	R/W	Hard maximum fast DAC code.
0x0E4	FAST_OUT_SAFE	R/W	Safe fast code during idle/fault.
0x0E8	FAST_SLEW_LIMIT	R/W	Optional maximum code change per update.
0x0EC	FAST_SAT_COUNT_LIMIT	R/W	Saturation duration before fault/relock.
0x0F0	FAST_OUT_CURRENT	R	Current fast DAC command.

11.5 Slow controller / recentering

Addr.	Name	R/W	Description
0x100	SLOW_CTRL_CONFIG	R/W	Enables scan, hold, slow integrator, recentering.
0x104	SLOW_BIAS	R/W	DC bias/center code.
0x108	SLOW_KI	R/W	Slow integral gain.
0x10C	SLOW_RECENTER_TARGET	R/W	Desired fast DAC center code.
0x110	SLOW_RECENTER_GAIN	R/W	Gain for keeping fast output centered.
0x114	SLOW_OUT_MIN	R/W	Hard minimum slow DAC code.
0x118	SLOW_OUT_MAX	R/W	Hard maximum slow DAC code.
0x11C	SLOW_OUT_SAFE	R/W	Safe slow DAC code during idle/fault.
0x120	SLOW_SLEW_LIMIT	R/W	Maximum slow code change per update.
0x124	SLOW_OUT_CURRENT	R	Current slow DAC command.

11.6 Ramp / scan generator

Addr.	Name	R/W	Description
0x140	RAMP_CONFIG	R/W	Triangle/sawtooth, direction, reset-on-start.
0x144	RAMP_MIN	R/W	Minimum scan code.
0x148	RAMP_MAX	R/W	Maximum scan code.
0x14C	RAMP_STEP	R/W	Code step per ramp tick.
0x150	RAMP_TICK_DIV	R/W	Divides clock/sample tick for ramp speed.
0x154	RAMP_CENTER	R/W	Center code for zoom scan.

0x158	RAMP_WIDTH	R/W	Width around center for zoom scan.
0x15C	RAMP_CURRENT	R	Current ramp code.
0x160	RAMP_CYCLE_COUNT	R	Completed scan cycles.

11.7 Trace capture

Addr.	Name	R/W	Description
0x180	TRACE_CONFIG	R/W	Enable, decimation, channel select.
0x184	TRACE_START	W	Start capture.
0x188	TRACE_LENGTH	R/W	Number of samples/pairs to capture.
0x18C	TRACE_DECIM	R/W	Capture every N th sample.
0x190	TRACE_STATUS	R	Busy, ready, overflow.
0x194	TRACE_WRITE_PTR	R	Internal write pointer.
0x198	TRACE_READ_ADDR	R/W	Address for host readback.
0x19C	TRACE_READ_DATA_X	R	Scan code at selected trace address.
0x1A0	TRACE_READ_DATA_Y	R	Error sample at selected trace address.

11.8 Robust autolock descriptor

Addr.	Name	R/W	Description
0x1C0	AUTOLOCK_CONFIG	R/W	Enable, robust mode, retry behavior.
0x1C4	AUTOLOCK_WINDOW_MIN	R/W	Selected scan window lower bound.
0x1C8	AUTOLOCK_WINDOW_MAX	R/W	Selected scan window upper bound.
0x1CC	AUTOLOCK_EXPECTED_MIN_X	R/W	Approximate selected feature minimum.
0x1D0	AUTOLOCK_EXPECTED_MAX_X	R/W	Approximate selected feature maximum.
0x1D4	AUTOLOCK_LOCK_X	R/W	Desired lock zero-crossing scan code.
0x1D8	AUTOLOCK_AMP_MIN	R/W	Minimum acceptable feature amplitude.
0x1DC	AUTOLOCK_WIDTH_MIN	R/W	Minimum acceptable feature width.
0x1E0	AUTOLOCK_WIDTH_MAX	R/W	Maximum acceptable feature width.
0x1E4	AUTOLOCK_SLOPE_SIGN	R/W	Expected lock slope sign.
0x1E8	AUTOLOCK_Y_OFFSET	R/W	Computed line offset.
0x1EC	AUTOLOCK_RETRY_LIMIT	R/W	Max retries before fault/idle.
0x1F0	AUTOLOCK_STATUS	R	Matched, failed, retry count, current phase.

11.9 Lock check / lock watch

Addr.	Name	R/W	Description
0x220	LOCK_CHECK_CONFIG	R/W	Enable post-lock check and lock watch.
0x224	LOCK_CHECK_DELAY	R/W	Samples to wait after engaging lock.
0x228	LOCK_CHECK_WINDOW	R/W	Samples used for check.
0x22C	LOCK_MAX_ERROR	R/W	Max allowed error while locked.

0x230	LOCK_MAX_FAST_ABS	R/W	Max allowed fast output deviation.
0x234	LOCK_MAX_SAT_COUNT	R/W	Saturation count before unlock.
0x238	RELOCK_POLICY	R/W	Hold, zoom rescan, wide rescan, fault.
0x23C	LOCK_WATCH_STATUS	R	Current lock-health metrics.

11.10 DAC configuration and safety

Addr.	Name	R/W	Description
0x260	DAC_CONFIG	R/W	DAC format, enable, test pattern, routing.
0x264	DAC_FAST_FORMAT	R/W	Offset-binary/two's-complement mapping if needed.
0x268	DAC_SLOW_FORMAT	R/W	Same for slow DAC.
0x26C	DAC_ROUTE	R/W	Normal, swapped, summed-to-fast, summed-to-slow, test.
0x270	DAC_FAST_CLAMP_HW_CODE	R/W	Digital mirror of expected analog clamp.
0x274	DAC_SLOW_CLAMP_HW_CODE	R/W	Digital mirror of expected analog clamp.
0x278	DAC_TEST_FAST	R/W	Manual fast DAC code in test mode.
0x27C	DAC_TEST_SLOW	R/W	Manual slow DAC code in test mode.

11.11 Latency instrumentation

Addr.	Name	R/W	Description
0x2A0	LATENCY_CONFIG	R/W	Enable timestamping/probes.
0x2A4	LATENCY_FAST_CYCLES	R	ADC CH0 valid to DAC FAST valid cycles.
0x2A8	LATENCY_ERROR_CALC	R	Error calculation latency.
0x2AC	LATENCY_PI	R	PI block latency.
0x2B0	LATENCY_LIMITER	R	Limiter latency.
0x2B4	LATENCY_TRACE	R	Trace path latency, non-fast.

12 Simulation and Verification Plan

12.1 Required fake models

- **Fake MTS signal model.** Generates a dispersive feature with configurable slope, width, offset, amplitude, noise, and drift.
- **Fake laser plant.** Converts fast and slow DAC commands into laser detuning.
- **Fake actuator saturation.** Models rails and slow/fast authority limits.
- **Fault injector.** Injects ADC rail, missing valid, saturation, wrong polarity, noisy trace, and feature disappearance.

12.2 Required system simulation

A full simulation should demonstrate:

```
wide scan
-> trace capture
-> selected feature descriptor loaded
-> zoom scan
-> FPGA robust autolock verifies feature
-> slow output holds center
-> PI state initialized
-> lock engages
-> error converges toward zero
-> slow recenter keeps fast DAC near center
-> lock watch detects healthy operation
```

12.3 Required plots

Every meaningful demo should produce plots of:

- scan code vs. time;
- MTS error trace vs. scan code;
- selected feature window and computed zero crossing;
- fast DAC code vs. time;
- slow DAC code vs. time;
- corrected error vs. time;
- lock state vs. time;
- saturation and fault flags vs. time.

12.4 Module-level tests

Testbench	Minimum tests
tb_adc_formatter.sv	input coding conversion, sign extension, boundary values.
tb_adc_guard.sv	overrange, stuck rail, missing valid, fault count.
tb_error_calc.sv	offset, setpoint, polarity, valid latency.
tb_pi_core.sv	P-only, I-only, PI, clamp, anti-windup, hold, reset, load.
tb_output_limiter.sv	boundary values, positive/negative overflow, saturation flags.
tb_ramp_scan.sv	triangle behavior, zoom bounds, no overshoot.
tb_trace_capture.sv	decimation, buffer write/read, overflow, ready flag.
tb_robust_autolock.sv	feature match, feature reject, retry, lock trigger timing.
tb_slow_recenter.sv	fast-output centering, slew limit, saturation.
tb_lock_fsm.sv	legal transitions, illegal commands, fault priority, explicit clear.
tb_lock_core_top.sv	full scan-to-lock simulation with fake plant.

13 Repository Structure

```
fpga_mts_lock/
  README.md
  LICENSE
  docs/
    00_project_mission.md
    01_mts_control_loop.md
    02_hardware_architecture.md
    03_fast_path_latency.md
    04_register_map.md
    05_autolock_algorithm.md
    06_safety_and_faults.md
    07_verification_plan.md
    08_reading_list.md
  rtl/
    common/
      sat_math.sv
      round_shift.sv
      sign_extend.sv
      stream_delay.sv
    adc/
      adc_formatter.sv
      adc_guard.sv
    dac/
      dac_fast_formatter.sv
      dac_slow_formatter.sv
    dsp/
      error_calc.sv
      pi_core.sv
      optional_compensator.sv
      lock_watch.sv
    control/
      ramp_scan.sv
      slow_recenter.sv
      output_limiter.sv
      fault_gate.sv
      lock_fsm.sv
    autolock/
      robust_autolock.sv
      peak_tracker.sv
      feature_matcher.sv
    bus/
      register_bank.sv
      register_defs_pkg.sv
    top/
      lock_core_top.sv
      board_top_stub.sv
  sim/
    tb_modules/
    tb_system/
    models/
      fake_mts_signal.py
      fake_laser_plant.py
    scripts/
      run_verilator.sh
```

```

    run_iverilog.sh
    plot_results.py
outputs/
  waveforms/
  csv/
  plots/
  reports/
    
```

14 Acceptance Checklist

A serious first handoff should satisfy the following:

- README explains the project and how to run simulations.
- All fast-path modules have documented width, signedness, reset behavior, and latency.
- Register map exists in both documentation and `register_defs_pkg.sv`.
- All actuator-facing outputs have programmable hard limits and safe codes.
- PI core supports reset, hold, load, saturation, and anti-windup.
- Ramp scan supports wide and zoom modes without overshoot.
- Trace capture records scan-code/error pairs and supports host readout.
- Robust autolock can verify a selected feature in simulation.
- Scan-to-lock handoff does not create an output jump.
- Lock check/watch detects unlock, saturation, and fault conditions.
- Slow recentering keeps fast output near a target center in simulation.
- Faults force safe outputs and require explicit clear.
- A full-system simulation demonstrates scan → trace → descriptor → zoom → autolock → lock → watch → slow recenter.
- Plots are generated automatically for review.

15 Papers, Datasheets, and Learning Resources

This section is intentionally practical. Each entry says what to learn and what not to copy blindly.

15.1 Spectroscopy locking and FPGA lockbox systems

Topic	Resource	Why it matters for POSM
-------	----------	-------------------------

Linien architecture and autolock	Wiegand et al., “Linien: A versatile, user-friendly, open-source FPGA-based tool for laser frequency stabilization,” arXiv:2203.02947. https://arxiv.org/abs/2203.02947	Best reference for user workflow: scan, click/drag line selection, automatic lock point selection, bare PID mode, and FPGA-side time-critical autolock. Do not copy the whole Red Pitaya/Migen/server stack.
Linien source	https://github.com/linien-org/linien	Useful for understanding feature selection and state behavior. Treat as inspiration, not code to port.
Simple FPGA laser lock	Jørgensen et al., “A simple laser locking system based on a field-programmable gate array,” Rev. Sci. Instrum. 87, 073106 (2016). https://arxiv.org/abs/1607.02860	Good first whole-system paper. Shows FPGA frequency modulation, lock-in detection, and PID for atom cooling/trapping use cases. POSM v1 differs by using analog demodulation for the required fast loop.
PyRPL	Neuhaus et al., “FPGA-based feedback control of quantum optics experiments with the open source software package PyRPL,” arXiv:2310.00086. https://arxiv.org/abs/2310.00086	Good reference for programmable digital filters, lock acquisition, gain adjustment, and diagnostics. Do not let the broad instrument-suite concept bloat POSM v1.
Open-source AMO servo	Leibrandt and Heidecker, “An open source digital servo for AMO physics experiments,” arXiv:1508.06319. https://arxiv.org/abs/1508.06319	Useful for latency/bandwidth thinking, GUI-controlled filters, MIMO control, transfer-function tools, and automatic lock acquisition.
Digital servo limitations	Yu et al., “Experimental limitations of an FPGA-based digital servo,” arXiv:1708.05892. https://arxiv.org/abs/1708.05892	Useful for understanding why latency limits feedback bandwidth and how digital phase compensation can help but not erase pure delay.
Many-channel FPGA controller	Schussheim and Gibble, “A multi-channel low-latency FPGA-based controller,” arXiv:2307.16008. https://arxiv.org/abs/2307.16008	Useful for low-latency digital PI/IIR habits, many-loop architectures, and slow correction concepts.

15.2 FPGA arithmetic, PI, filters, and verification

Topic/module	Resource	Use it for
Fixed-point arithmetic	Project F, “Fixed Point Numbers in Verilog.” https://projectf.io/posts/fixed-point-numbers-in-verilog/	Binary points, signed widths, bit growth, scaling, and why silent overflow is dangerous. Read before writing <code>pi_core.sv</code> .

Fixed-point PI	EmbeddedRelated, “How to Build a Fixed-Point PI Controller That Just Works.” https://www.embeddedrelated.com/showarticle/121.php	Practical PI saturation and anti-windup ideas. Translate concepts into RTL carefully.
Digital filters	Walker-Howell, “Parametric IIR Filtering on an FPGA.” https://ashrafi.sdsu.edu/PDF/Parametric_IIR_Filtering_on_an_FPGA.pdf	Useful background for IIR implementation, coefficient handling, and verification. POSM fast path should keep filters minimal and bypassable.
Streaming DSP habits	ZipCPU DSP/filter posts, e.g. https://zipcpu.com/dsp/2017/09/15/fastfir.html	Useful for valid signals, streaming design, test-driven DSP, and avoiding sloppy timing.
AXI-Lite later	ZipCPU, “Building an AXI-Lite slave the easy way.” https://zipcpu.com/blog/2020/03/08/easyaxil.html	Read only if converting the simple register bank into AXI-Lite. Do not overbuild the bus at the start.
Verilator	Verilator documentation. https://verilator.org/guide/latest/verilating.html	Linting, fast compiled simulation, and CI-style regression tests.
Formal/debug habits	ZipCPU tutorial. https://zipcpu.com/tutorial/	General FPGA debugging, assertions, and test discipline.

15.3 Device datasheets and hardware interface references

Device/topic	Resource	What to extract
AD9959 DDS	Analog Devices AD9959 datasheet/product page. https://www.analog.com/en/products/ad9959.html and https://www.analog.com/media/en/technical-documentation/data-sheets/ad9959.pdf	Channel synchronization, phase-offset register, frequency tuning word, amplitude control, SPI programming, IO update behavior.
AD9117 DAC	Analog Devices AD9114/AD9115/AD9116/AD9117 datasheet/product page. https://www.analog.com/en/products/ad9117.html	Dual DAC interface, timing, output coding, current-output behavior, transformer vs op-amp output options.
ADC3664 ADC	Texas Instruments ADC3664 product page/datasheet. https://www.ti.com/product/ADC3664	Dual-channel ADC interface, SLVDS timing, latency, input driver recommendations, VCM, AC/DC-coupled front ends.
CTL200 laser controller	Koheron CTL200 user guide. https://www.koheron.com/support/user-guides/ctl200/	DC and AC modulation input behavior, gains, input impedance, modulation ranges, safety/interlock behavior.

Red Pitaya/STEMlab references	Red Pitaya FPGA/docs. https://redpitaya.readthedocs.io/en/latest/intro.html	Useful for understanding existing lockbox ecosystems. Do not make POSM a Red Pitaya compatibility project.
-------------------------------	---	--

15.4 POSM-specific documents

- Previous POSM FPGA lock onboarding packet v2: useful for generic module specs, fixed-point discipline, and the original reference map.
- POSM MTS schematic: use for exact part numbers of photodiode/TIA, mixer, LPF, EOM driver, RF splitters/buffers, and protection networks. **TODO: Insert final schematic revision and part-number table once frozen.**
- POSM control/digital-twin disclosure notes: useful for understanding why lock status, faults, diagnostic streams, and recovery hooks matter for future POSMq integration.

16 Final Notes

The goal is not to write a pile of Verilog files. The goal is to create a simulated and eventually hardware-ready lock engine that can be trusted. A good first demonstration is not a GUI. A good first demonstration is a plot showing:

wide scan → selected descriptor → zoom scan → FPGA autolock → smooth PI handoff → stable lock → slow recentering.

If that simulation is clean, the hardware integration becomes much less scary.